

E-tiGo Pricing Engine

How It Works & What You Can Control / September 2026

This document walks through how pricing works in the E-tiGo platform, what each setting does, and the business decisions you can make through the admin dashboard without needing any code changes.

How Fares Are Calculated

Every fare in E-tiGo is calculated using a straightforward formula. When a passenger requests a ride, the system looks up the pricing configuration for their city and chosen vehicle class, gets the distance and estimated travel time for the route via Google Maps, and runs this calculation:

FARE FORMULA

Fare = max(Minimum Fare, Base Fare + (Distance x Per-km Rate) + (Duration x Per-minute Rate) + Waiting Charge) x Surge Multiplier

The **max()** part is important — it means the fare can never drop below whatever you set as the minimum fare. If a ride is very short (say, 0.5 km down the street), the calculated amount might be lower than the minimum, so the system bumps it up. This protects driver earnings on short trips.

The **Surge Multiplier** is applied after the base fare calculation. When no surge is active, the multiplier is 1.0 (no change). When surge is active (e.g. 1.5x during rush hour), the entire fare is multiplied.

What Each Rate Controls

RATE	WHAT IT DOES	BUSINESS IMPACT
Base Fare	Fixed amount charged on every trip, regardless of distance or time.	Sets your floor revenue per ride. Higher base = more predictable earnings per trip.
Per-km Rate	Charged per kilometre of the route.	Makes longer trips proportionally more expensive. The primary revenue lever for most rides.
Per-minute Rate	Charged per minute of estimated travel time.	Compensates drivers for time in traffic. Higher rates in congested cities make sense.
Minimum Fare	Absolute floor — fare can never go below this.	Protects driver economics on very short trips. Should cover at least fuel + opportunity cost.
Waiting Time Rate	Per-minute charge when the driver is waiting beyond the free period at pickup.	Compensates drivers for idle time. Applied after the free waiting window expires.
Free Waiting Minutes	Grace period before waiting charges begin (default: 5 minutes).	Balances passenger convenience with driver time. Industry standard is 3-5 minutes.

Example Fare Calculation

ABUJA — ECONOMY CLASS — 10 KM TRIP, 25 MIN ESTIMATED	
COMPONENT	AMOUNT
Base fare	₦600.00

COMPONENT	AMOUNT
Distance: 10 km x ₦250 /km	₦2,500.00
Time: 25 min x ₦40 /min	₦1,000.00
Subtotal	₦4,100.00
Surge: 1.0x (no surge)	₦4,100.00
Total fare	₦4,100.00

In this example, the calculated fare (~~₦4,100~~) is above the minimum fare (~~₦1,500~~), so the passenger pays ~~₦4,100~~. If this had been a 0.3 km trip with a 2-minute duration, the calculation would give ~~₦680~~ — below the minimum — so the fare would be ~~₦1,500~~ instead.

With Surge Pricing Active

ABUJA — ECONOMY CLASS — 10 KM TRIP, 25 MIN, 1.5x SURGE

COMPONENT	AMOUNT
Base fare	₦600.00
Distance: 10 km x ₦250 /km	₦2,500.00
Time: 25 min x ₦40 /min	₦1,000.00
Subtotal	₦4,100.00
Surge: 1.5x	₦6,150.00
Total fare	₦6,150.00

Recommended Default Rates

These are the recommended starting rates based on current fuel costs and market analysis (September 2026):

Abuja

SETTING	ECONOMY	COMFORT	PREMIUM
Base Fare	₦600	₦800	₦1,200
Per-km Rate	₦250	₦350	₦500
Per-minute Rate	₦40	₦55	₦80
Minimum Fare	₦1,500	₦2,000	₦3,000
Waiting Rate	₦50/min	₦65/min	₦100/min
Free Waiting	5 min	5 min	5 min

Lagos

SETTING	ECONOMY	COMFORT	PREMIUM
Base Fare	₦600	₦800	₦1,200
Per-km Rate	₦250	₦350	₦500
Per-minute Rate	₦40	₦55	₦80
Minimum Fare	₦1,500	₦2,000	₦3,000
Waiting Rate	₦50/min	₦65/min	₦100/min
Free Waiting	5 min	5 min	5 min

These rates ensure drivers profit on every trip at current fuel costs (₦1,430/litre). Example: a 10 km, 25 min Abuja Economy trip yields ₦4,100 — with fuel cost around ₦1,590 for a sedan, that leaves ₦2,510 for the driver before platform commission.

You can adjust these at any time through the admin dashboard. They are defaults — set them as starting points and tune based on your market data.

Waiting Time Charges

When a driver arrives at the pickup location and the passenger isn't ready, waiting time charges kick in — but only after a free grace period. Here's how it works:

WAITING CHARGE FORMULA

Waiting Charge = max(0, Actual Wait - Free Waiting Minutes) x Waiting Time Rate

Every pricing config includes a **free_waiting_minutes** value (defaults to 5 minutes). The driver waits without the passenger being charged anything for that period. After the free period expires, every additional minute is charged at the **waiting_time_rate**.

Example: Waiting Time on an Abuja Economy Ride

DRIVER WAITS 8 MINUTES — 5 MIN FREE, RATE: ₦50/MIN

Total wait time	8 min
Free waiting period	-5 min
Chargeable minutes	3 min
Waiting charge: 3 min x ₦50/min	₦150.00

How estimates work: When passengers request a fare estimate, they see the waiting time policy (free minutes and per-minute rate) so they know the rules before booking. The actual waiting charge is only applied when the ride is completed, based on how long the driver actually waited.

Surge Pricing

Surge pricing lets you increase fares during periods of high demand. The system applies a multiplier (e.g. 1.5x) to the calculated fare. You control when surge is active, how high it goes, and what triggers it.

How It Works

The surge multiplier is applied **after** the base fare calculation (including the minimum fare check):

Final Fare = Base Fare (after min check) x Surge Multiplier

When passengers see a fare estimate during an active surge, they see: - The total fare (already multiplied) - A "1.5x surge" indicator so they know pricing is elevated - The surge rule name (e.g. "Morning Rush Hour")

Three Types of Surge Rules

TYPE	HOW IT WORKS	BEST FOR
Manual	You toggle it on/off from the admin dashboard. Active immediately when enabled.	Rain, accidents, special events, emergencies — anything you can see happening and want to respond to quickly.
Time-Based	Fires automatically based on a schedule: which days of the week, start time, and end time.	Rush hours (e.g. Mon-Fri 7-9am and 5-8pm), weekend night surges (Fri-Sat 10pm-2am).

TYPE	HOW IT WORKS	BEST FOR
Demand-Based	Fires automatically when the demand-to-supply ratio exceeds a threshold you set.	Real-time demand spikes. <i>(Currently a placeholder — will activate once real-time driver availability tracking is live.)</i>

What You Control

For each surge rule, you set:

SETTING	WHAT IT DOES	CONSTRAINTS
City	Which city the rule applies to.	Required.
Vehicle Class	Optionally scope to a specific vehicle class. Leave empty = applies to all classes in the city.	Optional.
Name	A label you'll see in the dashboard (e.g. "Morning Rush Hour", "Heavy Rain").	Required.
Multiplier	How much to multiply the fare by. 1.5 = 50% increase, 2.0 = double.	1.00 to 5.00.
Priority	When multiple rules match at the same time, the highest priority wins.	0-100 (higher = wins).
Effective From / Until	Date range when the rule can be active.	"Until" is optional — omit for no expiry.
Active	Master on/off switch.	Toggle anytime.

Time-Based Schedule

For time-based rules, you configure: - **Days of week:** 1 = Monday, 7 = Sunday. Example: `[1, 2, 3, 4, 5]` for weekdays. - **Start time:** When surge begins (24-hour format, e.g. `07:00`). - **End time:** When surge ends (e.g. `09:00`).

Overnight spans work too — setting start `22:00` and end `06:00` covers the late-night period.

Priority & Conflict Resolution

If a city has multiple active surge rules that all match at the same time (say, a rush hour rule AND a manual rain rule), the system picks the one with the **highest priority number**. If two rules have the same priority, it picks the one with the **higher multiplier**.

This means you can layer rules: set a weekday rush hour at priority 5 (1.3x) and a manual "heavy rain" at priority 10 (1.8x). On a rainy Monday at 8am, both match — the rain rule wins because it has higher priority.

Example Surge Scenarios

Abuja Morning Rush Hour: - Type: Time-based - Days: Monday-Friday - Time: 07:00-09:00
- Multiplier: 1.3x - Priority: 5

Heavy Rain (Manual): - Type: Manual - Multiplier: 1.8x - Priority: 10 - Toggle on when it starts raining, toggle off when it stops.

Weekend Night Premium: - Type: Time-based - Days: Friday, Saturday - Time: 22:00-03:00
- Multiplier: 1.5x - Priority: 5 - Vehicle class: Premium only

What Passengers See

When surge is active, the fare estimate response includes:

```
"surge": {
  "active": true,
  "multiplier": 1.5,
  "rule_name": "Morning Rush Hour"
}
```

The mobile app should display a clear surge indicator so passengers can decide whether to wait or accept the higher fare. When no surge is active, the multiplier is 1.0 and the fare is unchanged.

Pricing by City and Vehicle Class

Pricing is configured per **city + vehicle class** combination. This means you can set completely different rates for:

- Different cities (Lagos pricing vs. Abuja pricing vs. Port Harcourt pricing)
- Different vehicle classes within the same city (Economy vs. Comfort vs. Premium in Lagos)

This gives you fine-grained control. You might want Economy rides in Abuja to start at ₦600, but Premium rides to start at ₦1,200 — and those same classes in Lagos might have completely different numbers based on local economics.

Key point: You need to create a pricing config for every city + vehicle class combination you want to offer rides in. If a city has 3 active vehicle classes, that's 3 pricing configs to set up. Without a pricing config, that vehicle class won't appear in fare estimates for that city.

Versioning — How Pricing Changes Work

Pricing configs are **versioned and append-only**. When you create a new pricing config for the same city + vehicle class, the system doesn't overwrite the old one — it creates a new version alongside it. This is deliberate, and it gives you two important capabilities:

Price History

You can always look back and see what your rates were at any point in time. This is useful for auditing, resolving disputes, and understanding revenue trends. The admin dashboard shows you all versions with their effective dates.

Scheduled Price Changes

Every pricing config has an **effective_from** date. You can create a new pricing config today that won't kick in until next Monday. The system will keep using the current rates until that date, then automatically switch to the new ones. No manual intervention needed on the day.

In-progress ride protection: When a passenger books a ride, the system takes a snapshot of the pricing at that moment and locks it to that ride. If you change pricing while someone is mid-trip, their fare stays on the rate they were quoted. This prevents disputes and builds passenger trust.

Cross-City Rides & Edge Cases

The pricing engine handles several real-world scenarios automatically. Here's how they work and what the passenger sees.

Cross-City Rides

When a passenger books a ride from one city (e.g. Abuja) to another city (e.g. Lagos), the system:

1. **Detects the destination city** automatically using the destination coordinates
2. **Applies pickup city pricing** — the ride uses Abuja rates, not Lagos rates
3. **Warns the passenger** with a message like: *"This is a cross-city ride from Abuja to Lagos. Pickup city (Abuja) pricing applies for this trip."*

This is the industry standard approach (used by Uber, Bolt, and others). The pickup city's rates apply for the entire ride because that's where the driver started and where the platform's operational costs are based.

Example: A passenger in Abuja books a ride to Lagos (850 km). The Economy fare would be: ₦600 base + (850 × ₦250/km) + (600 min × ₦40/min) = ₦237,100 — using Abuja rates.

Destination Outside Service Area

If the destination doesn't fall within any of E-tiGo's active cities, the estimate still works — it uses the pickup city's pricing. The app shows a warning: *"Your destination is outside our current service areas. The driver may not be able to accept return trips from the destination."*

This lets passengers take rides to locations you haven't launched in yet, while managing expectations about driver availability for the return trip.

Long-Distance Rides

Routes over 100 km automatically include a warning that the fare is estimated and may vary. The fare is still calculated normally — this is just a heads-up for the passenger that real-world conditions (traffic, road closures, detours) may affect the final fare more on longer routes.

Short Rides & Minimum Fare

For very short rides (e.g. 0.3 km), the calculated fare might be much lower than the minimum fare. The system automatically bumps it to the minimum. This protects driver earnings — a trip that calculates to ₦680 would be charged at ₦1,500 (the minimum) instead.

What the App Sees

When edge cases are detected, the API response includes extra fields:

```
"warnings": [
  "This is a cross-city ride from Abuja to Lagos. Pickup city (Abuja) pricing applies for",
  "This is a long-distance ride (over 100 km). Fare is estimated based on route distance a",
],
"cross_city": {
  "destination_city_id": 2,
  "destination_city_name": "Lagos",
```

```
"pickup_city_name": "Abuja"  
}
```

The mobile app should display these warnings clearly before the passenger confirms the booking. For standard same-city rides, these fields are not included.

Business Decisions You Can Make

Here are the main levers you have through the admin dashboard, and when you might want to pull each one:

Adjust Rates by City

If a particular city has higher fuel costs, more traffic, or a different cost of living, you can set rates accordingly. You might charge more per km in a city with expensive fuel and more per minute in a city with heavy congestion.

Differentiate Vehicle Classes

Premium and luxury classes should carry a meaningful price premium. A rule of thumb: Comfort is typically 1.3-1.5x Economy, Premium is 2-2.5x. EV classes could be priced at a slight discount to incentivise green rides, or at a premium if positioned as a luxury offering.

Schedule Rate Changes

Planning a rate increase? Create the new pricing config with a future effective date. Want to run a promotional week with lower fares? Set lower rates effective Monday and higher rates effective the following Monday. Both will apply automatically.

Set Minimum Fares Strategically

The minimum fare is your most important tool for driver retention. If it's too low, drivers lose money on short trips and stop accepting them. If it's too high, passengers walk instead of booking. Check your average trip distance — minimum fare should be attractive enough that drivers don't skip short rides.

Configure Waiting Time Policy

You control both the free waiting period and the per-minute rate. The default is 5 minutes free (Bolt uses 4). Setting this too low frustrates passengers; too high and drivers lose money waiting around. You can adjust per city and vehicle class.

Use Surge Pricing

Activate surge during periods of high demand to balance supply and demand. A moderate surge (1.2-1.5x) encourages more drivers to come online while slightly reducing ride requests — bringing the market toward equilibrium. Use time-based rules for predictable peaks and manual rules for unexpected events.

Launch a New City

When expanding to a new city: create the city in the admin panel, enable the vehicle classes you want to offer there, then create pricing configs for each city + vehicle class combination. The city goes live for passengers as soon as pricing is in place.

What's Coming Next

The pricing engine is built to be extended. Here's what's planned for future phases:

FEATURE	WHAT IT WILL DO	STATUS
Surge Pricing	Dynamic multiplier during high demand. Three rule types: manual, time-based, demand-based.	Done
Google Maps Integration	Fare estimates use actual road distances and real-time traffic data via Google Maps Distance Matrix API.	Done
Cross-City Ride Detection	Automatic detection of cross-city rides with warnings and pickup city pricing. Handles destination outside service area and long-distance rides.	Done
Demand-Based Surge (Live)	Demand-based surge rules will fire automatically once real-time driver availability tracking is wired in.	In Progress

FEATURE	WHAT IT WILL DO	STATUS
Promo Integration	Promotional discounts applied on top of the calculated fare. The promo engine is a separate module that hooks into fare calculation.	Next Sprint
Tier Discounts	Automatic booking fee discounts for passengers in higher loyalty tiers. Configured through the gamification settings.	Planned

Quick Reference

Admin Actions — Pricing

ACTION	WHERE IN DASHBOARD
Create new pricing	Admin → Pricing → New Config
View all pricing history	Admin → Pricing (filter by city / vehicle class)
Check current active rate	Admin → Pricing → Current (select city + class)
Schedule a future rate	Create new pricing with a future "Effective From" date

Admin Actions — Surge Pricing

ACTION	WHERE IN DASHBOARD
Create surge rule	Admin → Surge Rules → New Rule
View all surge rules	Admin → Surge Rules (filter by city)
Toggle surge on/off	Admin → Surge Rules → Toggle Status
Check current multiplier	Admin → Surge Rules → Current Multiplier (select city)
Set rush hour schedule	Create time-based rule with days + times

ACTION	WHERE IN DASHBOARD
Emergency surge	Create manual rule, toggle on immediately

E-tiGo Backend — Pricing & Surge Engine Documentation Questions? Reach out and we'll walk you through any of this in more detail.